# Отчёт по лабораторной работе №4

**Web-программирование: «Реализация простого сайта на Django»**

**Выполнил:** Клименко Богдан Владимирович, ИСУ №408790, группа K3340
**Тема: Реализация клиентской части приложения средствами Vue.js**
**Вариант: 19 – Учёт животных в зоопарке**

## 1 Цель работы:

Овладеть практическими навыками разработки клиентской части веб-приложения на Vue.js, интеграции с backend-API, реализации аутентификации и авторизации, а также документирования разработанных интерфейсов.

## 2 Ход выполнения:

2.1 Используемые технологии

Фронтенд: Vue 3 (сборка Vite), Vue Router, Pinia, Vuetify 3, Axios

Бэкенд: Django + Django REST Framework (из лабораторной работы №3)

Аутентификация: JWT (djangorestframework-simplejwt, djoser)

Документирование: MkDocs

3. Ход выполнения работы

3.1. Настройка CORS на бэкенде

Для разрешения запросов с фронтенда на порту 5173 в Django-проекте были выполнены следующие действия:

Установлен пакет django-cors-headers.

В settings.py добавлено приложение corsheaders, middleware CorsMiddleware, а также список разрешённых источников:

CORS_ALLOWED_ORIGINS = ["http://localhost:5173", "http://127.0.0.1:5173"]

### 3.2. Создание Vue-проекта и установка зависимостей

Проект создан с помощью npm create vue@latest с опциями: Vue Router, Pinia, без TypeScript. Установлены дополнительные пакеты:

npm install axios vuetify@next @mdi/font

### 3.3. Реализация аутентификации (вход, регистрация, профиль)

- Созданы компоненты LoginView.vue, RegisterView.vue, ProfileView.vue.
- В хранилище user.js (Pinia) реализованы методы login, register, logout, fetchUser.
- JWT-токен сохраняется в localStorage и автоматически подставляется в заголовки запросов через перехватчик Axios.
- В router/index.js добавлена защита маршрутов, требующих аутентификации.

### 3.4. Реализация интерфейсов для работы с животными (CRUD)

- Создан сервис animals.js для взаимодействия с API (методы getAll, getById, create, update, delete).

- Компоненты:
  - AnimalListView.vue – отображение списка животных в виде карточек.
  - AnimalDetailView.vue – детальная информация о животном.
  - AnimalCreateView.vue – форма добавления нового животного.
  - AnimalEditView.vue – форма редактирования.
- Все формы стилизованы с помощью Vuetify и поддерживают выбор из справочников (отделы, вольеры, зоны обитания), загружаемых с бэкенда.

### 3.5. Реализация отчётов (кастомные запросы)

В соответствии с вариантом созданы страницы для отображения аналитической информации:

- DepartmentAnimalCountView.vue – количество животных в каждом отделе (GET /api/v1/departments/animal-count/).
- EmptyEnclosuresView.vue – список пустых вольеров (GET /api/v1/enclosures/empty/).
- LeaseReportView.vue – отчёт по аренде животных (GET /api/v1/zoos/lease-report/).

### 3.6. Подключение Vuetify и стилизация

- Vuetify 3 подключён через плагин plugins/vuetify.js.
- Все компоненты используют готовые элементы интерфейса (карточки, кнопки, таблицы, формы).
- Навигационная панель реализована в App.vue с адаптивным меню.

### 3.7. Документирование интерфейсов с MkDocs

- Создана документация с помощью MkDocs, описывающая структуру проекта, основные компоненты, маршруты и примеры запросов к API.
- Статическая документация собрана в папку site.

---

## 4. Скриншоты работы приложения

*Рисунок 1 – Страница входа*

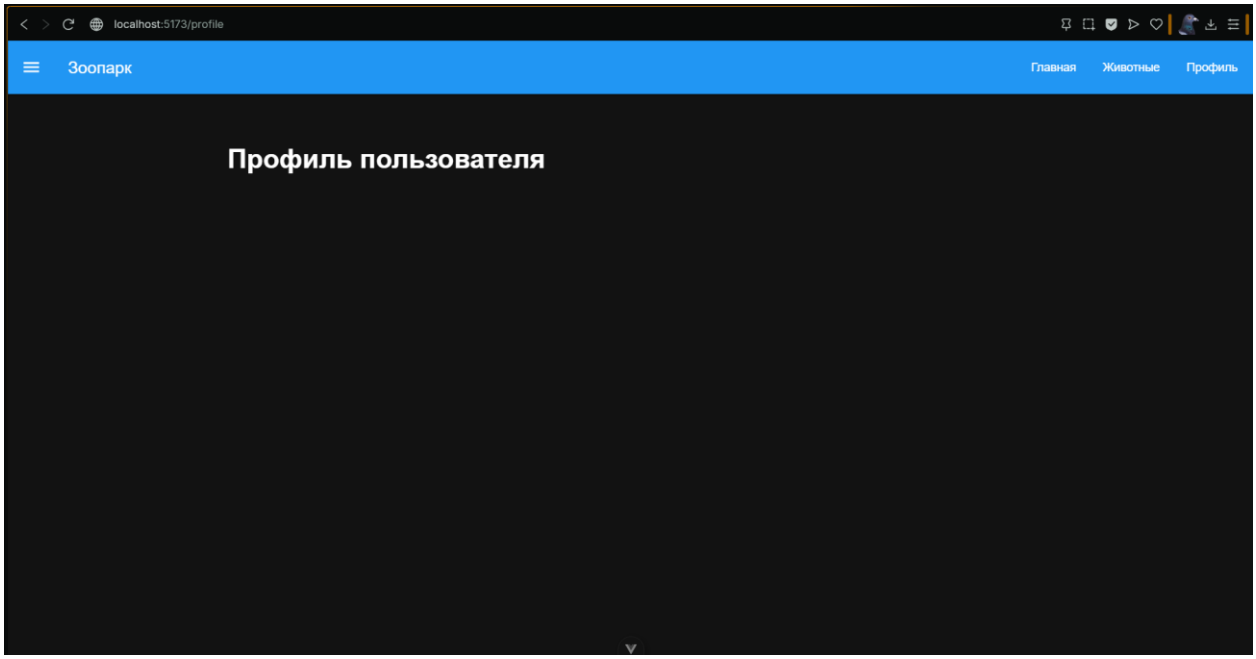


*Рисунок 2 – Список животных*

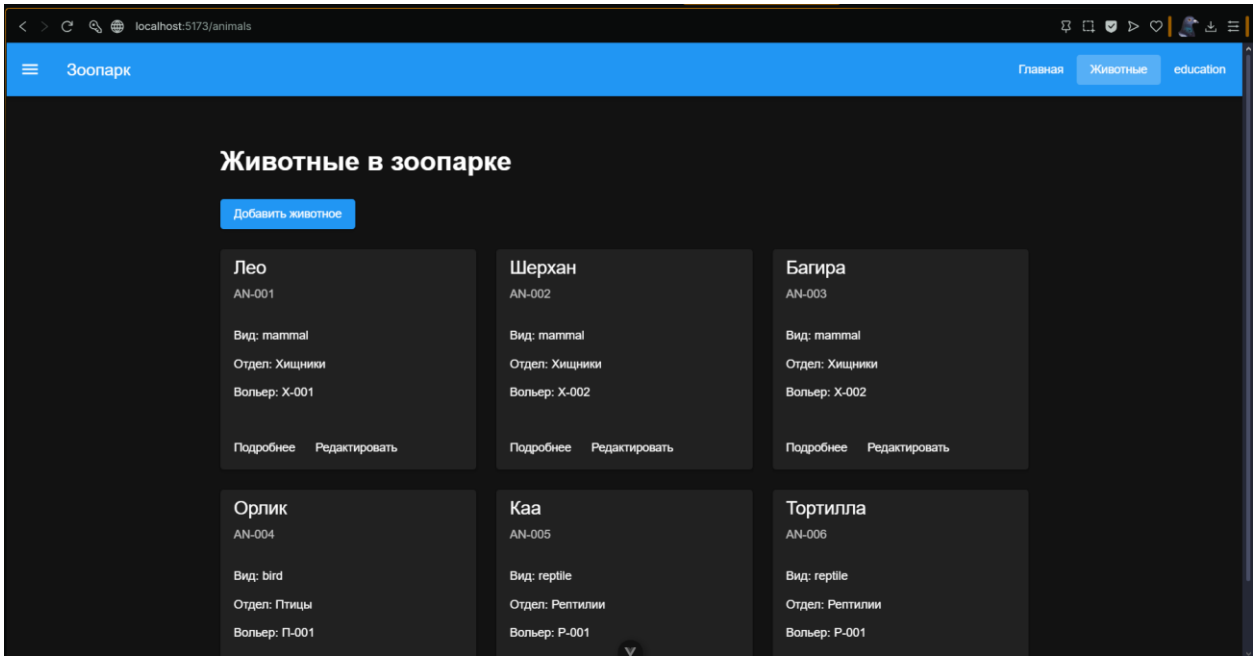

*Рисунок 3 – Детальная страница животного*

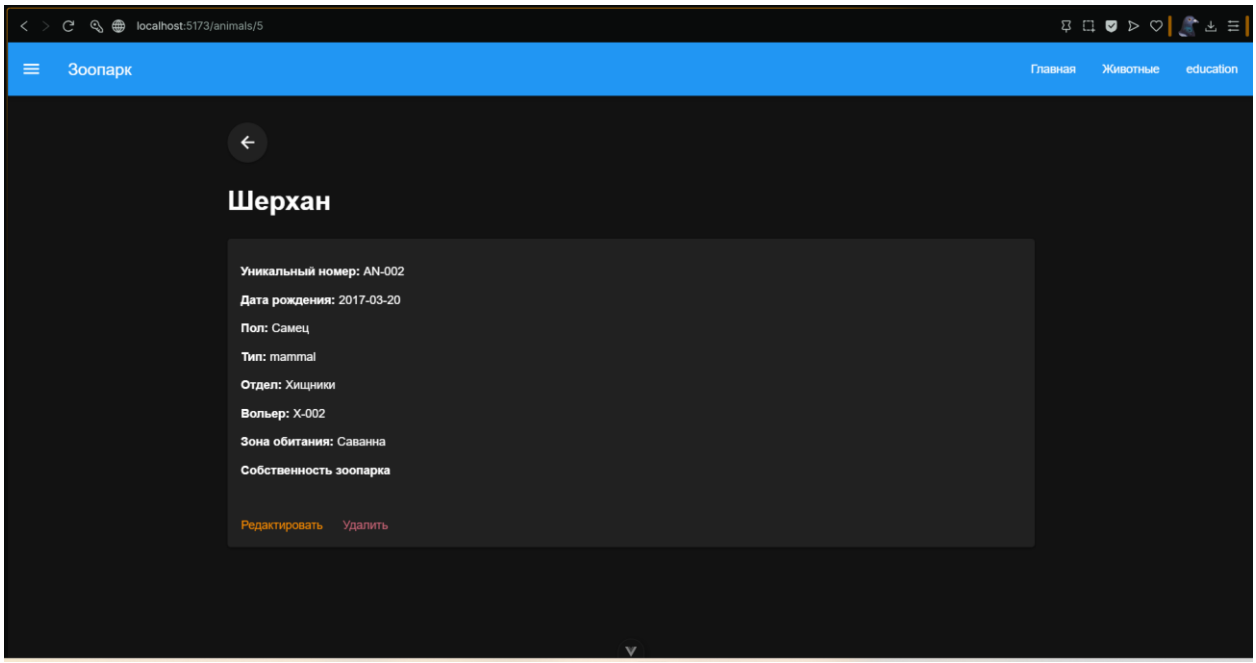


*Рисунок 4 – Форма редактирования животного*

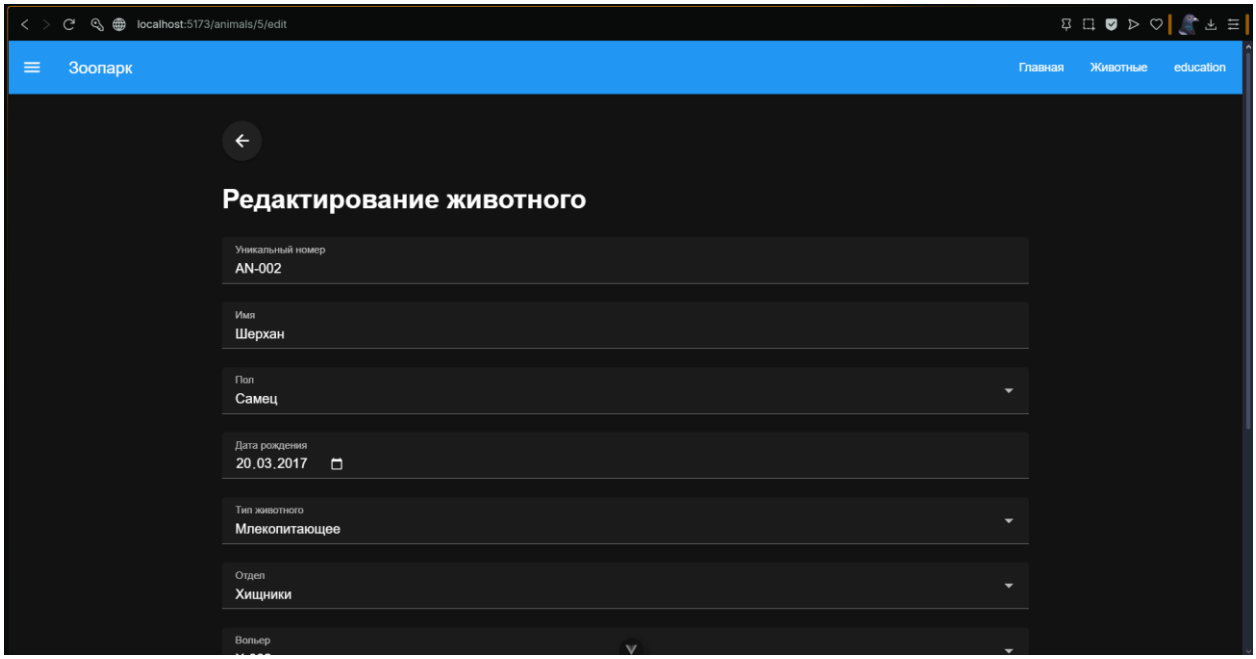

*Рисунок 5 – Отчёт по количеству животных в отделах*

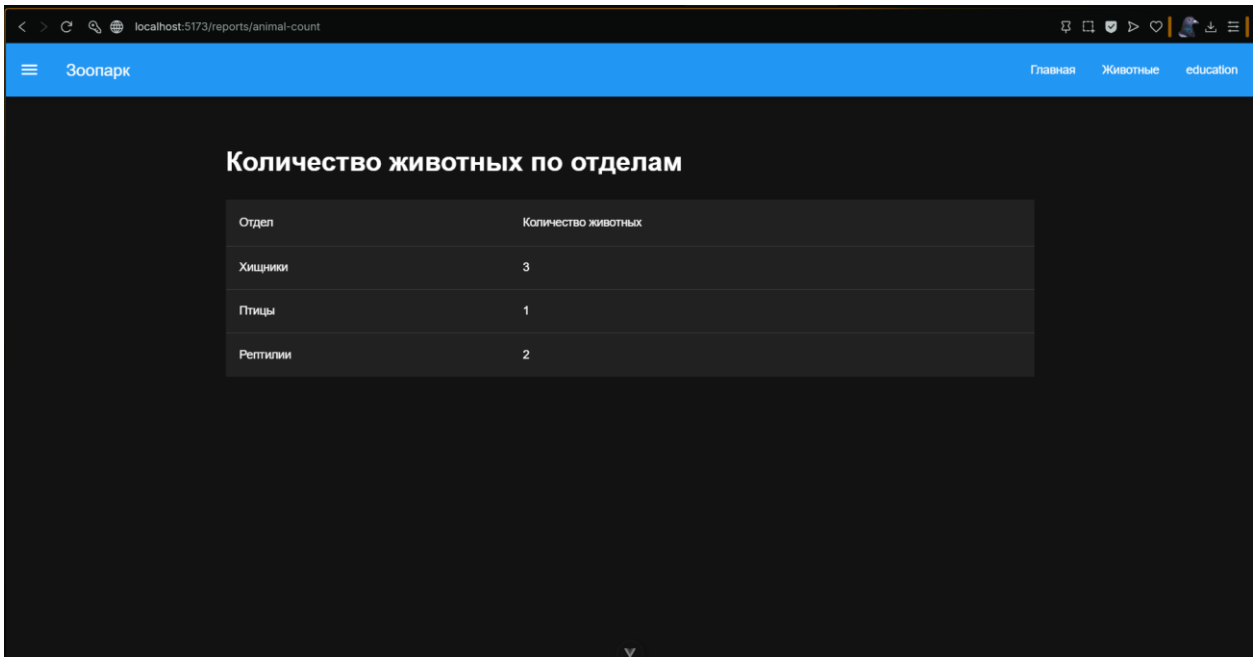


*Рисунок 6 – Отчёт по пустым вольерам*

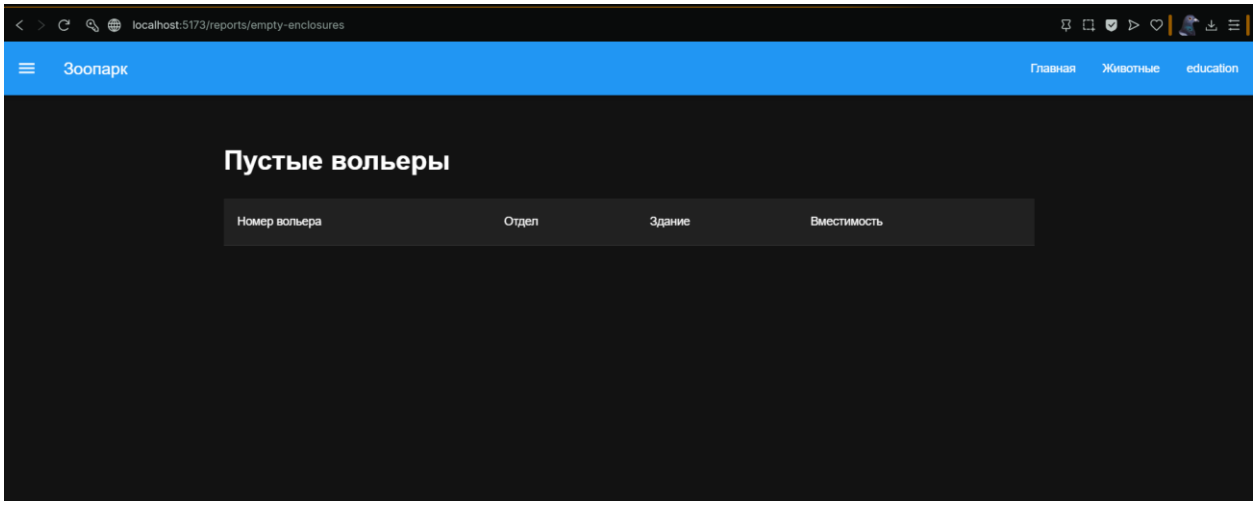

*Рисунок 7 – Отчёт по аренде*

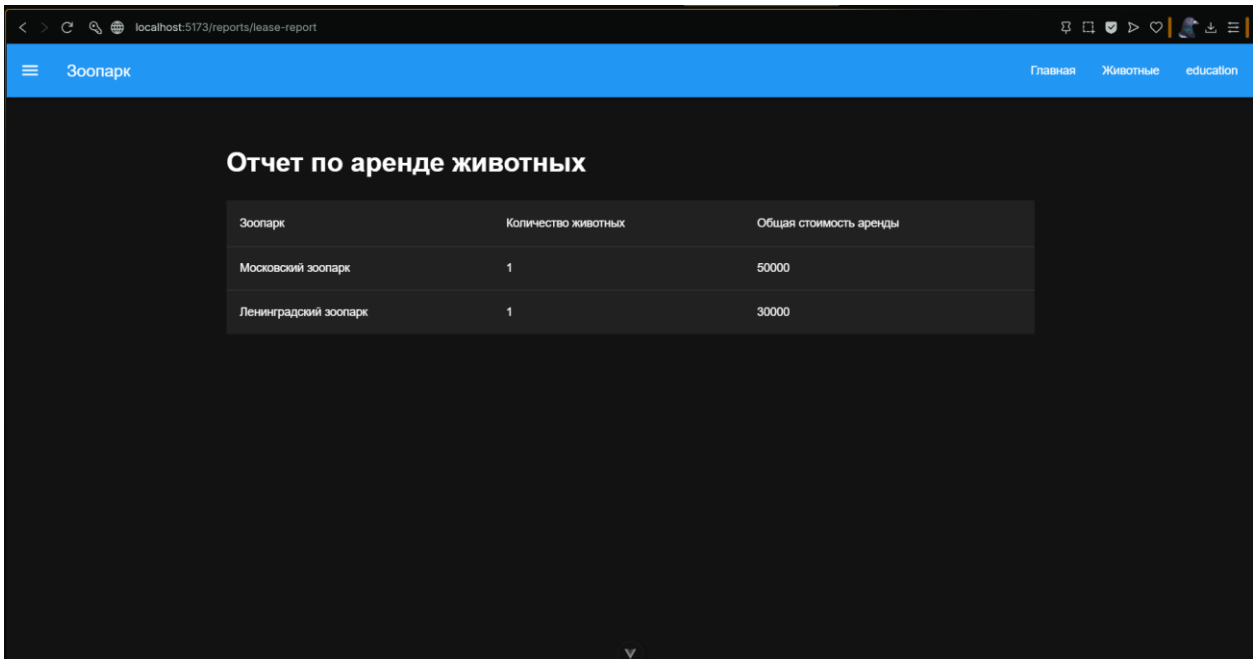


*Рисунок 8 – Документация MkDocs*

## 5. Выводы

В результате выполнения лабораторной работы было разработано полноценное клиентское приложение на Vue.js, взаимодействующее с REST API, созданным в лабораторной работе №3. Реализованы все необходимые функции: аутентификация, CRUD для животных, три кастомных отчёта. Интерфейс выполнен с использованием библиотеки Vuetify, что обеспечило современный и адаптивный дизайн. Настроена работа с JWT-токенами, решены проблемы CORS. Подготовлена документация MkDocs. Полученные навыки позволяют создавать клиент-серверные приложения с разделением фронтенда и бэкенда.